

- [Assignment 1: Constraint Mapping Across Data Sources](#)
 - [0. Matched Results \(CSV\):](#)
 - [1. Overview](#)
 - [2. Methodology](#)
 - [3. Match Performance & Statistics](#)
 - [4. Key Insights](#)
 - [5. Next Steps for Production](#)
- [Assignment 2: Bus-level Load Prediction Pipeline](#)
 - [0. Model Predictions \(2025 Calendar Year High-Resolution\)](#)
 - [1. Executive Summary & Evaluation Metrics](#)
 - [Performance Summary Table](#)
 - [2. Methodology & Model Architecture](#)
 - [What Model Was Used](#)
 - [What Baseline Was Compared Against](#)
 - [What Features Were Created](#)
 - [How Future Data Usage Was Avoided \(No Data Leakage\)](#)
 - [3. Performance Analysis & Strategy Comparison](#)
 - [Where the Model Works Well](#)
 - [Where It Performs Poorly](#)
 - [Strategy Comparison: Direct Bus vs. Zone Forecast + Bus Share](#)
 - [4. Future Improvements \(With More Time\)](#)
 - [5. AI Usage Logs & Chat History](#)
 - [Process Explanation](#)
 - [Chat History](#)

Assignment 1: Constraint Mapping Across Data Sources

0. Matched Results (CSV):

[Download Matched Constraints \(CSV\)](#)

1. Overview

This project maps power grid constraints across **PJMISO Market** , **Dayzer** , and **Panorama** data. By combining domain heuristics with fuzzy matching, the pipeline successfully aligns varying naming conventions to provide a unified view of congestion risk.

2. Methodology

- **Domain-Informed Normalization:** Standardized text and implemented a targeted heuristic for Panorama to extract core constraint names hidden within parentheses (e.g., extracting **CHIC_AVE 138 KV CHI-PRA2** from the raw string).
- **Fuzzy Matching:** Leveraged **thefuzz** library. Used **token_set_ratio** for Panorama to handle out-of-order strings, and **partial_ratio** for Dayzer to bridge specific market constraints with broader "Interface" definitions.

3. Match Performance & Statistics

Based on the 5,230 PJMISO Market constraints:

- **Panorama:** Median Score: **97.0** | 75th Percentile: **100.0** | Unmatched: **26.29%**
- **Dayzer:** Median Score: **89.0** | 75th Percentile: **96.0** | Unmatched: **23.40%**

4. Key Insights

1. **High Accuracy on Overlaps:** The high median scores (97 for Pano, 89 for Dayzer) strongly validate the heuristic rules and matching engine for constraints that exist across systems.
2. **Topological Granularity Explains the ~25% Gap:** The unmatched rates reflect physical model differences, not algorithmic failures. PJMISO includes localized, low-voltage nodes (69kV/138kV). Commercial models (Dayzer/Panorama) focus on backbone transmission (345kV/500kV) and major regional interfaces.
Returning **NaN** safely prevents false positives.

5. Next Steps for Production

- **Cross-Contingency Validation:** Use the **Contingency** column as a secondary verification layer to mathematically boost or penalize borderline facility match scores.
- **Topology-Based Graph Matching:** Decompose Dayzer Interfaces into "FROM" and "TO" nodes to map directly against PJM bus topology, reducing reliance on pure string similarity.

Assignment 2: Bus-level Load Prediction Pipeline

0. Model Predictions (2025 Calendar Year High-Resolution)

Due to GitHub's file size limitations (100MB), the full high-resolution hourly forecast outputs for the calendar year 2025 are securely hosted on Google Drive.

You can download the final predictions directly via the links below:

- **Task 1: Day-Ahead Forecast Results**
 - [Download Day-Ahead Predictions \(CSV\)](#)
- **Task 2: Next-Month Forecast Results**
 - [Download Next-Month Predictions \(CSV\)](#)

1. Executive Summary & Evaluation Metrics

This report summarizes the performance of our machine learning load forecasting pipeline evaluated on 2025 ground-truth data.

Performance Summary Table

Evaluation Scale	Forecast Horizon	MAE	RMSE	WMAPE
Zone-Level(Aggregated)	Next-Month Forecast	665.62 MW	1157.55 MW	9.50%
Zone-Level(Aggregated)	Day-Ahead Forecast	422.10 MW	719.91 MW	6.02%
Bus-Level(Individual Nodes)	Next-Month Forecast	4.52 MW	14.74 MW	29.66%
Bus-Level(Individual Nodes)	Day-Ahead Forecast	4.25 MW	14.19 MW	27.71%

2. Methodology & Model Architecture

What Model Was Used

We implemented a **Gradient Boosted Decision Tree (GBDT)** framework (LightGBM/XGBoost) due to its high accuracy on tabular data, capacity to handle categorical features (`bus_id`, `zone_id`), and robust performance with non-linear relationships.

What Baseline Was Compared Against

The model was evaluated against a **Historical Multi-Granularity Baseline** , which calculated the historical average load grouped by `bus_id`, `hour_of_day`, and `day_of_week`, augmented by naive "same hour last week" and "same hour last year" benchmarks.

What Features Were Created

- **Temporal Features:** Hour of day, day of week, month, and a binary weekend flag.
- **Lagged Features:** Historical load values from matching hours ($t - 1$ day, $t - 7$ days) to capture autoregressive patterns.

- **Rolling Aggregations:** 24-hour and 7-day rolling mean/standard deviation of load values to capture momentum and shifting baselines.
- **Spatial Categoricals:** `bus_id` and `zone_id` target encodings to model structural grid geography.

How Future Data Usage Was Avoided (No Data Leakage)

To prevent data leakage, we enforced a strict time-series separation rule:

- For **Day-Ahead Forecasts**, features at day t only used historical data up to day $t - 1$. Lagged features and rolling windows never crossed into the target day.
 - For **Next-Month Forecasts**, lag features and rolling averages were strictly restricted to data available prior to the start of the target month ($t - 30$ days or greater). Training data was partitioned chronologically, ensuring the model never trained on validation-year data.
-

3. Performance Analysis & Strategy Comparison

Where the Model Works Well

- **Zone-Level Aggregations:** Achieving a WMAPE of **6.02%** for day-ahead and **9.50%** for next-month forecasts is highly robust.
- **Short Horizon Windows:** The model leverages recent data effectively, making Day-Ahead projections significantly sharper than mid-term projections.

Where It Performs Poorly

- **Individual Bus Nodes:** Bus-level forecasting shows high error rates (WMAPE between **27% and 29%**). Individual buses are highly volatile, prone to local consumer switching behavior, localized maintenance, and random industrial load spikes that lack clear patterns.

Strategy Comparison: Direct Bus vs. Zone Forecast + Bus Share

- **Next-Day Forecasts: Direct Bus Forecasting** performs slightly better here. Because the information is fresh, the model can accurately track immediate localized momentum and recent bus-specific operational trends.
 - **Next-Month Forecasts: Zone Forecast + Bus Share** works significantly better. Predicting thousands of highly volatile individual bus series over a 30-day horizon introduces compound errors. Instead, predicting the stable, aggregated macro-trend of the Zone and applying historical bus allocation fractions protects the model from chaotic local variations over long periods.
-

4. Future Improvements (With More Time)

1. **Incorporate Localized External Drivers:** Integrating bus-level industrial calendars or weather telemetry (temperature/humidity) would directly target and reduce the 27%+ error rate at the nodal level.
 2. **Optimize Pipeline Processing:** Transitioning the chunked evaluation code to multi-threaded frameworks like Polars or Dask would reduce the massive 67-minute evaluation window down to under 5 minutes.
-

5. AI Usage Logs & Chat History

Process Explanation

AI tools were heavily utilized as a development partner throughout this assignment:

1. **Code Troubleshooting & Optimization:** When the initial evaluation script caused memory allocation crashes due to a 143-million-row merge, the AI refactored the pipeline to use a **Memory-Optimized Index-Join Architecture** ($O(1)$ lookup complexity) and downcasted datatypes to save physical RAM.

2. **Jupyter Kernel Resolution:** When the system crashed and locked network ports (Timeout Waiting for Ports), the AI provided step-by-step diagnostic instructions to purge the local Jupyter runtime cache folder (%appdata%\jupyter\runtime) and terminate ghost zombie processes via terminal task killing.

Chat History

Power Market Constraint Mapping Task - Google Gemini